Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- RLE ile görüntü sıkıştırma (slayt başlığı “Görüntü Sıkıştırma”, ekrandan, 00:10:00)
  - Dijital görüntü piksellerden oluşur. Bir piksel 1 bit (siyah-beyaz, B&W), birkaç bit (gray scale), 8 bit (color table, 0-255) ya da 24 bit (true color: red, green, blue kanallarının her biri 8 bit) olabilir. Slayttaki tablo: 4 bit 16 renk, 7 bit 128 renk, 8 bit 256 renk.
  - RLE metinde başarısızdır, çünkü metinde art arda tekrar eden karakter azdır (boşluklar dışında). Görüntüde, özellikle siyah-beyaz görüntüde art arda tekrar eden piksel blokları çok olduğu için başarılıdır.
  - RLE, JPEG gibi birçok sıkıştırma yönteminin ve WinRAR, WinZip gibi araçların ara adımıdır.
- Bitmap okuma ve tarama (ekrandan, 00:20:00)
  - Pikseller bitmap içinde scan line’lara yerleşir. İlk piksel sol üst köşedeki, son piksel sağ alt köşedeki noktadır.
  - RLE ile sıkıştırmada nereden başlandığı önemlidir. Rastgele seçilmiş bir pikselin komşularının kendisiyle aynı renkte olma ihtimali zayıftır; rastgele bir noktadan başlamak uzun tekrarları kaçırabilir. Bu yüzden genelde sol üstten başlanır.
  - Sıkıştırılacak dosya sıkıştırılmamış olmalıdır: bitmap (BMP) uygundur; JPEG ya da TIFF zaten sıkıştırılmış biçimlerdir.
  - Bitmap dosyasının başında header bulunur (genişlik, yükseklik, piksel başına bit sayısı gibi). RLE yalnız data kısmına uygulanır; header sıkıştırılmış verinin başına eklenir, yoksa veri açılamaz.
- Siyah-beyaz resimde RLE (ekrandan, 00:27:30)
  - Compressor bitmap’i satır satır, sütun sütun ya da zigzag tarar.
  - İlk okunan pikselin beyaz olduğu kabul edilir. İlk piksel beyaz değilse output stream’e “0 adet beyaz” yazılır. Sonra sayılar sırayla beyaz, siyah, beyaz… run uzunluklarıdır; renk kodu yazılmaz.
  - Örnek: 17 beyaz, 1 siyah, 55 beyaz piksel output stream’e `17 1 55` olarak yazılır.
  - Output stream’in başına çözünürlük yazılır. Hocanın örneğinde 72 scan line ve satır başına 256 piksel: çözücü run’ları toplayarak (17, 18, 73, …) 256’ya ulaşınca bir sonraki scan line’a geçer.
  - Header’da tarama biçimi de tutulur. Hocanın örneği: `00` satır satır, `01` sütun sütun, `10`/`11` zigzag.
  - Sıkıştırılmış stream’in uzunluğu şeklin karmaşıklığına bağlıdır: detay arttıkça oran kötüleşir.
  - Uniform bir alanda RLE için sıkıştırma oranı yaklaşık olarak $\frac{\text{çevre uzunluğu}/2}{\text{alan içindeki piksel sayısı}}$ olur. Her scan line şekle bir yerden girip bir yerden çıktığı için $2\times$ scan line sayısı şeklin çevre uzunluğunu verir.
- Renkli palet örneği: 15 x 11 piksel, 16 renk (ekrandan, 00:30:00 ve 00:34:25)
  - Sıkıştırmasız: $15 \times 11 \times 4 = 660$ bit.
  - Kullanılan RLE versiyonu: her run için önce run değeri, sonra renk kodu. Run değeri en çok 15 olabileceği (satır 15 piksel) için o da 4 bitle yazılır.
  - Özgün resim: 134 değer, $134 \times 4 = 536$ bit.
  - Renk sayısı azaltılmış resim: 86 değer, $86 \times 4 = 344$ bit (ekrandan, 00:36:40).
  - Renk sayısı artırılmış, renklerin sürekli değiştiği resim: 311 değer, $311 \times 4 = 1244$ bit; sıkıştırma değil genişletme olur (ekrandan, 00:38:10).

- RLE kayıpsız ve kayıplı
  - Anlatılanların hepsi kayıpsızdır. Komşu renkler arasındaki fark gözün algılayamayacağı kadar küçükse (örneğin 15 yerine 14) bu renk değiştirilip kayıplı sıkıştırma yapılabilir. Kısa run'lar ihmal edilip komşu pikselin rengine çevrilebilir.
  - Tıbbi görüntüde kayıplı sıkıştırma yapılamaz; küçük bir nokta (ör. bir kist) kaybolabilir.
- RLE'nin farklı versiyonları (ekrandan, 00:40:00 ve 00:50:00)
  - Örnek veri (8 bit gray scale bitmap): `12 12 12 12 12 12 12 12 12 35 76 112 67 87 87 87 5 5 5 5 5 5 1 ...` Run değeri ve renk kodu yazılırsa: `9 12 35 76 112 67 3 87 6 5 1 ...` Sorun: bilgisayar hangi sayının run değeri olduğunu bilemez.
  - I-a. Görüntü 128 gri seviyeyle (0-127, 7 bit) sınırlıysa her byte'ın en soldaki biti, byte'ın gri değer mi tekrar sayısı mı olduğunu gösterir.
  - I-b. Gri seviye 0-255 ise 255 değeri 254 olarak kodlanır ve 255 özel escape kodu olur. 255'ten sonra gelen iki byte tekrar sayısı ve piksel değeridir: `255 9 12 35 76 112 67 255 3 87 255 6 5 1 ...`
  - II. Her byte'ın ne olduğunu gösteren bir flag biti kullanılır; bu bitler 8'lik gruplar hâlinde 1 byte'ta toplanır ve ardından gelen 8 byte'tan önce gönderilir (her 8 byte için fazladan 1 byte): `10000010 9 12 35 76 112 67 3 87 100... 6 5 1 ...` Bazı uygulamalar byte yerine word kullanıp 16 byte'ı yorumlar; hocaya göre byte daha anlamlıdır.
  - III. True color resimde her piksel 3 byte'tır (R, G, B). Her palet için ayrı RLE uygulanır: `(171 85 34) (172 85 35) (172 85 30) (173 85 33)` pikselleri `(171 172 172 173)`, `(85 85 85 85)`, `(34 35 30 33)` dizilerine ayrılır (ekrandan, 00:51:20).
- Tarama biçimleri (ekrandan, 01:00:00)
  - Satır satır, sütun sütun, rotate (yılan gibi: satır sonunda başa dönmeden alttaki satırdan geri gelerek) satır ve sütun, zigzag.
  - Komşu piksellerin renkleri birbirine daha yakın olduğu için rotate okuma daha uzun run'lar verir. Bir pikselin 8 komşusu, resmin uzak bir köşesindeki pikselden daha ilişkilidir.
  - Hangisinin daha iyi olduğu resmin içeriğine bağlıdır; dikey kalın çizgiler varsa sütun sütun okumak daha iyidir.
  - Zigzag okuma en idealidir. JPEG resmin tamamını değil, 8x8 (64 piksel) ya da 16x16'lık bloklarını zigzag okur.
- 8x8 siyah-beyaz örnek (ekrandan, 01:02:40)
  - Satır satır: 4 satır `1 1 1 2 1 1 1 <eoln>` ($8 \times 4 = 32$), 2 satır `7 1 <eoln>` ($3 \times 2 = 6$), 2 satır `0 8 <eoln>` ($3 \times 2 = 6$); toplam 44 byte.
  - Sütun sütun: `5 1 1 1 <eoln>` ($5 \times 3 = 15$), `0 4 1 1 1 1 <eoln>` ($7 \times 4 = 28$), `4 4 <eoln>` ($3 \times 1 = 3$); slaytta toplam 44 byte yazıyor.
  - Ders sonrası not: Sütun toplamı $15 + 28 + 3 = 46$ byte eder; slayttaki 44 büyük olasılıkla bir yazım hatasıdır (python ile kontrol edildi).
- RLE özeti (ekrandan, 01:10:00)
  - Kayıpsız bir yöntemdir (istenirse kayıplı kullanılır), aynı değere sahip uzun verilerde en yüksek başarıyı verir, kodlaması kolaydır, çeşitli varyasyonları vardır, hızlı çalışır, karışık veride negatif sıkıştırma yapar.
- Bilgi teorisi (ekrandan, 01:40:00)
  - Bir olayın gerçekleşme ihtimali ne kadar düşükse taşıdığı bilgi o kadar fazladır; ihtimali yüksek olay az bilgi taşır (olasılık ile bilgi ters orantılı).
  - Öz bilgi: $I(A) = \log_x \frac{1}{P(A)} = -\log_x P(A)$. Bilgisayarda iki durum (var/yok) olduğu için $x = 2$ alınır; sonuç bit cinsindendir.
  - Sıkıştırmadaki karşılığı: sık geçen sembol az bitle, nadir sembol çok bitle kodlanır. Hocanın örneği: 20 A ve 5 J, ASCII'de $20 \times 8 = 160$ ve $5 \times 8 = 40$ bit. A 3 bit, J 12 bit olursa $60 + 60 = 120$ bit; J'de kaybedilen, A'da kazanılandan azdır.
  - Hilesiz para: $P = 1/2$, $I = 1$ bit. Hileli para: $P(\text{yazı}) = 1/8 \Rightarrow I = 3$ bit, $P(\text{tura}) = 7/8 \Rightarrow I \approx 0{,}193$ bit. Pratikte kesirli bit yoktur.
- Entropi
  - Bağımsız olaylar kümesi S için ortalama öz bilgi: $H = \sum_i P(A_i)I(A_i) = -\sum_i P(A_i)\log_2 P(A_i)$. Olasılıklar 0-1 arasında olduğundan logaritma negatiftir; öndeki eksiyle H hep pozitif çıkar.
  - Entropi, sembolleri kodlamak için olması gereken ideal ortalama bit uzunluğunu

(bit/sembol) verir.
  - Shannon: en iyi kayıpsız sıkıştırmada kodlanan sembollerin ortalama bit uzunluğu kaynağın entropisine eşit olmalıdır; ancak bu ideal sıkıştırma pratikte mümkün olmamaktadır (ekrandan, 01:50:00).
  - Örnek: $a_1 = 0{,}49$, $a_2 = 0{,}25$, $a_3 = 0{,}25$, $a_4 = 0{,}01$ için $H \approx 1{,}57$ bit (ekrandan, 01:52:50). 4 sembol sabit 2 bitle kodlanır.
  - Ortalama mesaj uzunluğu S ile entropi arasındaki fark artıklıktır (redundancy): $R = |S - H|$. Örnekte $R = 2 - 1{,}57 = 0{,}43$ bit. R sıfıra ne kadar yakınsa seçilen kodlama o kadar iyidir.
  - Hesap makinesinde 2 tabanında logaritma: $\log_2 x = \frac{\ln x}{\ln 2} = \frac{\log_{10} x}{\log_{10} 2}$ (ekrandan, 02:00:00).
- Sayı dizisi örneği ve residual sequence (ekrandan, 02:00:00 ve 02:10:00)
  - Dizi: `1 2 3 2 3 4 5 4 5 6 7 8 9 8 9 10` (16 sayı). $P(1) = P(6) = P(7) = P(10) = 1/16$, diğer altı sayı $2/16$. $H = 3{,}25$ bit. 10 farklı sembol sabit 4 bitle kodlanırsa $R = 0{,}75$, iyi değil.
  - Residual sequence: ilk eleman yazılır, sonra ardışık elemanların farkları: `1 1 1 -1 1 1 1 -1 1 1 1 1 1 -1 1 1`. $P(1) = 13/16$, $P(-1) = 3/16$, $H \approx 0{,}70$ bit. 1 bitle kodlanır, $R = 0{,}3$.
  - Alıcının modeli bilmesi gerekir: $x_n = x_{n-1} + r_n$. Header'da verinin residual olup olmadığı bir byte ile belirtilir.
  - Bu bir dönüşümdür; veri sıkıştırmaya daha uygun hâle getirilir. Ardışık değerler arasındaki değişimin küçük olduğu verilerde (ör. 10 dakikada bir ölçülen oda sıcaklığı) başarılıdır.
  - Parametreleri n'ye göre değişmeyen modellere statik, değişenlere dinamik model denir; dinamik modeller sıkıştırma ve açma zamanı açısından daha maliyetlidir.
- Vize sorusu örneği: blok alma (ekrandan, 02:17:50)
  - Dizi: `1 2 1 2 3 3 3 3 1 2 3 3 3 3 1 2 3 3 1 2` (20 sembol). $P(1) = P(2) = 1/4$, $P(3) = 1/2$, $H = 1{,}5$ bit/sembol; 20 sembol için 30 bit.
  - İkişerli bloklar (`12` ve `33`) alınırsa $H = 1$ bit/blok. `12` için 0, `33` için 1 verilirse $S = 1$, $R = 0$.
  - Ders sonrası not: Slaytta ve anlatımda $P(12) = P(33) = 1/4$ yazıyor; 10 blokta her biri 5 kez geçtiği için doğrusu $P(12) = P(33) = 1/2$'dir. $H = 1$ bit/blok sonucu bu olasılıklarla çıkar (10 blok, 10 bit).
- Veri modelleri (ekrandan, 02:20:00)
  - Fiziksel model: verinin fiziksel özelliği (ses gürültülü ortamda mı alındı, ölçüm verisi mi) kullanılır. Residual örneği de bir fiziksel modeldir.
  - Olasılık modeli: her sembolün birbirinden bağımsız üretildiği kabul edilir. Kaynak hakkında hiçbir bilgi yoksa kullanılan ve sembolleri eşit olasılıklı sayan hâline ignorance model denir. Olasılıklar kullanım sıklığından hesaplanırsa probability model olur.
  - Markov model: sembollerin olasılıkları birbirine bağımlı hesaplanır (A.A. Markov). Kayıpsız sıkıştırmada kullanılır, "Discrete Time Markov Chain" olarak da bilinir (ekrandan, 02:30:00).
  - k. dereceden Markov modeli: $P(x_n \mid x_{n-1}, \dots, x_{n-k}) \approx P(x_n \mid x_{n-1}, \dots, x_{n-k}, \dots)$. Geçmişteki k sembol bilgisi tüm geçmişe yaklaşık olarak eşittir. Pratikte k genelde 1 ya da 2 seçilir.
- Markov modelde entropi (ekrandan, 02:40:00)
  - Siyah-beyaz resimde iki durum vardır: S_w (beyaz) ve S_b (siyah). Her durumdan aynı duruma kalma ya da diğer duruma geçme olasılığı vardır.
  - $H = \sum_i P(S_i) H(S_i)$; her durumun entropisi o durumdan çıkan geçiş olasılıklarıyla hesaplanır.
  - Örnek: $P(S_w) = 0{,}8$, $P(S_b) = 0{,}2$, $P(w \mid b) = 0{,}3$, $P(b \mid w) = 0{,}01$.
  - Olasılıksal model: $H = -(0{,}8 \log_2 0{,}8 + 0{,}2 \log_2 0{,}2) = 0{,}722$ bit.
  - Markov model: $H(S_b) = 0{,}881$, $H(S_w) = 0{,}081$, $H = 0{,}2 \times 0{,}881 + 0{,}8 \times 0{,}081 = 0{,}241$ bit.

Hocanın Özellikle Vurguladığı Kısımlar

- RLE'de başarı resmin içeriğine ve seçilen RLE versiyonuna bağlıdır
  - Aynı boyuttaki resimde renk sayısı azaldıkça bit sayısı düşer (536 → 344), renkler sürekli değişince 1244 bite çıkar ve sıkıştırma yerine genişletme olur.
- Sıkıştırılmış dosyanın açılabilmesi için nasıl sıkıştırıldığı header'da olmalıdır
  - Resmin boyutu, run ve renk için ayrılan bit sayısı, tarama biçimi (satır, sütun, zigzag) header'a yazılır.
- Run değerini renk kodundan ayırmak RLE versiyonlarının temel problemidir
  - Çözümler: soldaki biti flag yapmak, 255'i escape kodu yapmak, 8 byte için bir flag byte'ı göndermek.
- JPEG içinde RLE neden başarılı?
  - JPEG RLE'yi özgün renklere değil, önce bir dönüşüm uygulanmış ve belli bir sayı aralığına indirgenmiş değerlere uygular. Açarken önce RLE çözülür, sonra dönüşümün tersi alınır.
- Entropi ideal ortalama bit uzunluğudur, redundancy sıfıra ne kadar yakınsa yöntem o kadar iyidir
  - Veriyi olduğu gibi kodlamak yerine dönüşüm (residual) uygulamak ya da tekrar eden örüntüleri tek sembol almak entropiyi düşürebilir.
- Model seçimi yer ve zaman arasındaki bir tercihtir
  - Olasılıksal model daha kolay ve hızlıdır; Markov model daha karmaşıktır ama daha düşük entropi, yani daha iyi sıkıştırma verir. Yer kazanmak önemliyse Markov model, zaman önemliyse daha basit model seçilir.

Kısa Tekrar Notları

- RLE görüntüde başarılı, metinde başarısızdır; siyah-beyaz resimde en iyisidir.
- Siyah-beyaz RLE: ilk run beyaz kabul edilir; ilk piksel siyahsa önce 0 yazılır. Renk kodu yazılmaz, yalnız run uzunlukları yazılır.
- RLE bitmap'in data kısmına uygulanır; header sıkıştırılmaz ama çıktıya eklenir.
- Uniform alanda sıkıştırma oranı ≈ (çevre/2) / (alan içindeki piksel sayısı).
- 15x11, 16 renk: 660 bit → 536, 344 ya da 1244 bit (renk dağılımına göre).
- RLE versiyonları: 7 bitlik veride soldaki bit flag; 8 bitte 255 escape (255 → 254); 8 byte için 1 flag byte'ı; true color'da R, G, B ayrı ayrı.
- Tarama: satır, sütun, rotate satır/sütun, zigzag. JPEG 8x8 bloklarda zigzag okur.
- RLE kayıpsızdır; kısa run'lar ihmal edilirse kayıplı olur. Tıbbi görüntüde kayıp kabul edilmez.
- $I(A) = -\log_2 P(A)$; $H = -\sum P(A_i) \log_2 P(A_i)$; $R = |S - H|$.
- $\log_2 x = \log_{10} x / \log_{10} 2$.
- Residual sequence: $r_n = x_n - x_{n-1}$, geri dönüş $x_n = x_{n-1} + r_n$; 3,25 bit → 0,70 bit.
- Blok alma: `1 2 1 2 3 3 ...` dizisi tek tek 1,5 bit/sembol, ikişerli 1 bit/blok.
- Modeller: fiziksel, olasılık (ignorance, probability), Markov (k. derece, pratikte $k = 1$ ya da 2).
- Markov entropisi: $H = \sum P(S_i)H(S_i)$; örnekte 0,722 bit (bağımsız) yerine 0,241 bit.

Detaylı Açıklamalar

- **Görüntü türleri ve RLE.** Siyah-beyaz bir resimde piksel 1 bitle (0 siyah, 1 beyaz) ifade edilir. Gri tonlu resimlerde 3 bit 8, 4 bit 16 farklı ton verir; 8 bitlik color table'da renkler 0-255 arasıdır. True color resimde red, green, blue paletlerinin her biri 0-255 arasında değer alır ve 8 bit yer kaplar, toplam 24 bit eder. Renk sayısı arttıkça art arda aynı rengin gelme ihtimali azalır. Siyah-beyaz resimde ise özel olarak hazırlanmamışsa piksel piksel siyah-beyaz değişimi olmaz; uzun siyah ve beyaz bloklar oluşur, bu yüzden RLE çok daha başarılıdır. True color resimde her palet ayrı RLE'den geçirilir: önce bütün red değerleri, RLE bitince green, sonra blue. Hoca kanalları scan line bazında iç içe bağlamanın işi karıştıracağını ve hata olasılığını artıracağını söyledi. Açarken üç paletten gelen değerler birleştirilerek tek piksel elde edilir.
- **Başlangıç noktası ve tarama.** Hoca gökyüzü, bulut ve deniz olan bir resim çizerek anlattı: sol üstten başlanırsa gökyüzü uzun süre mavi okunur, sonra beyaz bulut, sonra yine

mavi gelir. Rastgele bir noktadan başlanırsa kısa mavi bir parçadan hemen beyaza geçilir ve tekrarlar kaçırılır. Siyah dikey bir çizgi kolon kolon okunurken başından başlanırsa uzun bir siyah run elde edilir. Aynı mantıkla tarama yönü de önemlidir: dikey kalın siyah çizgiler sütun sütun okunursa uzun siyah run'lar oluşur. Rotate okumada satır sonunda başa dönülmez, bir alttaki satırdan geri gelinir; böylece fiziksel olarak komşu pikseller ardışık okunur. Hocanın tahtadaki F harfi örneğinde (16 piksel genişlik) satır satır okuma `0 10 6, 0 10 6, ...` verir; rotate satır okumada ilk satırın sonundaki 6 beyaz ile ikinci satırın sonundaki 6 beyaz birleşip 12 beyaz, ikinci satırın 10 siyahı ile üçüncü satırın başındaki 2 siyah birleşip 12 siyah olur. Sütun sütun okuma tahtada `0 12 0 12 0 2 3 2 6 ...` olarak yazıldı (ekrandan, 01:00:00).

- **Uniform alanda sıkıştırma oranı.** Tek renkli bir şeklin üzerinden örneğin 50 scan line geçiyorsa her scan line şekle bir noktadan girip bir noktadan çıkar; $50 \times 2 = 100$ şeklin çevre uzunluğunu verir. RLE'nin sıkıştırma oranı kabaca bu çevrenin yarısının şeklin içindeki piksel sayısına oranıdır. Şekil ne kadar karmaşıksa (çok detaylı oda fotoğrafı) oran o kadar kötüdür; büyük siyah bir artı işareti gibi basit şekillerde iyi sonuç alınır.
- **Renkli palet örneğinin yorumu.** 16 renkli bir paletle ($2^4 = 16$) 15 x 11 piksellik resim her piksel 4 bit olduğundan 660 bittir. Run değeri + renk kodu versiyonunda tek pikselin olduğu yerde 1 değer yerine 2 değer yazılır (kayıp), ama 15 aynı pikselin olduğu satırda 15 değer yerine 2 değer yazılır (kazanç). Hoca, run değerleri de en fazla 15 olacağı için 4 bit yetmesini bu resme özgü bir varsayım olarak belirtti; run 8 bitle tutuluyorsa hesap değişir. Renkler azaltıldığında daha uzun run'lar oluştuğu için 344 bit, renkler sürekli değiştiğinde 1244 bit elde edildi.
- **Ders sonrası not.** Hoca 00:38:30'da "siyah beyaz resimlerde RL kullanmam abesle iştigal" dedi; bağlam ve slayt, renklerin sürekli değiştiği (karışık) resimlerde RLE kullanmanın anlamsız olduğunu gösteriyor. Hoca hemen sonra siyah-beyaz resimlerde RLE'nin çok iyi sonuç verdiğini de söylemişti.
- **JPEG'de RLE.** Hoca, JPEG içinde RLE'nin özgün renklere değil, önce bir dönüşüm (altyazıda "kosan transformu ya da veror transformu") uygulanıp belli bir sayı aralığına indirgenmiş değerlere uygulandığını söyledi. Böylece sanki daha az renkli bir resme RLE uygulanmış olur.
- **Kayıplı RLE.** Hoca, JPEG olarak kaydederken run değerinin sorulabildiğini ve örneğin 2'den kısa run'ların ihmal edilip komşu pikselin rengine çevrilebildiğini anlattı. Ders sonrası not: Yaygın resim düzenleyicilerde JPEG kaydederken sorulan ayar büyük olasılıkla kalite (quality) ayarıdır; kısa run'ları ihmal etme fikri ise kayıplı RLE'nin genel mantığıdır.
- **Ödevin akademik içeriği.** En az iki siyah-beyaz BMP resminin data kısmı kendi yazılan RLE koduyla sıkıştırılıp açılır; açılan resim özgünüyle aynı değilse sıkıştırmada hata vardır. Hazır bitmap okuma kütüphanesi kullanılabilir, ama RLE kodu hazır alınamaz. Header boyutu sıkıştırılmış boyuta eklenir (hocanın örneği: 64 byte veri + 32 byte header = 96 byte). Satır (rotate row), sütun (rotate column) ve zigzag okuma ile elde edilen boyutlar bir tabloda karşılaştırılır; zigzag tüm resimde ya da resim parçalara bölünerek uygulanabilir. Kullanılan RLE versiyonu belirtilir.
- **Bilgi teorisi sezgisi.** Hocanın örnekleri: yazın 28-30 derecede deniz kenarında kar yağması çok düşük ihtimallidir; yağarsa çok bilgi taşıyan bir olaydır. Bir haftadır yağmur yağarken öğlen de yağmur yağması çok olasıdır ve bilgi taşımaz. Bir sayfada A harfinin J'den çok geçtiği saymadan bilinir; A'nın bilgisi az, J'nin bilgisi çoktur. ASCII her sembolü 8 bitle kodlar; olasılıklar farklıyken hepsini 8 bit kodlamak sıkıştırma yapmamak demektir.
- **Entropi örneklerinde hesap.** $H = -(0{,}49 \log_2 0{,}49 + 2 \times 0{,}25 \log_2 0{,}25 + 0{,}01 \log_2 0{,}01) \approx 1{,}57$ bit; 4 sembol için 2 bit gerektiğinden $R = 0{,}43$. Sayı dizisinde $H = 4 \cdot \frac{1}{16} \cdot 4 + 6 \cdot \frac{2}{16} \cdot 3 = 1 + 2{,}25 = 3{,}25$ bit. Residual dizide $H = -(\frac{13}{16} \log_2 \frac{13}{16} + \frac{3}{16} \log_2 \frac{3}{16}) \approx 0{,}696$ bit. Değerler python ile doğrulandı. Hoca slayttaki $P(1) = 1/6$ değerinin $1/16$ olması gerektiğini derste düzeltti (ekrandan, 02:00:00).
- **Ders sonrası not (Shannon-Fano).** Hoca "Shannon Fano diye bir adam var; Shannon adı, Fano soyadı" dedi. Shannon-Fano kodlaması büyük olasılıkla iki ayrı kişinin, Claude

Shannon ile Robert Fano'nun adını taşır. Hoca bir sonraki konunun Shannon-Fano yöntemi olacağını söyledi.

- **Olasılık modeli ve Markov modeli sezgisi.** Hoca masasındaki eşyalarla anlattı: mouse, defter, zil, bardak gibi nesnelerin olasılıkları birbirinden bağımsız sayılırsa (ör. iki mouse varsa 2/10) olasılık modelidir. Laptop olduğu için mouse, su olduğu için bardak bulunması gibi bağımlılıklar hesaba katılırsa Markov modeline geçilir. Metinde A'nın olasılığı kendinden önce B, C ya da D gelmesine göre farklı hesaplanır. Hoca örnek olarak $P(a) = 0{,}5$, $P(a \mid b) = 0{,}7$, $P(a \mid c) = 0{,}6$ ve $P(a \mid bd)$ için de 0,69 civarı bir değer verdi ; geçmiş uzatıldıkça değişim çok küçük olur (ekrandan, 02:30:00). Örneğin 1000 sembollük geçmiş kullanılırsa ilk 1000 sembol hiç kodlanamaz ve olasılık sayısı patlar; bu yüzden k genelde 1 ya da 2'dir.
- **Ders sonrası not (Markov örneği).** Hoca örneği anlatırken "Hidden Markov Model kullanıyorum" dedi; örnekteki model durumları doğrudan gözlenen (siyah/beyaz) birinci dereceden Markov modelidir, gizli (hidden) Markov modeli değildir. Ayrıca anlatımda $P(b \mid w)$ "0,1" ve $P(w \mid w)$ "0,09" ya da "0,99" diye geçiyor; slaytta $P(b \mid w) = 0{,}01$ ve $H(S_w)$ hesabında $0{,}99$ kullanılmıştır. Hoca da her iki durumun entropisini sözlü olarak "0,81" diye okudu; slaytta $H(S_b) = 0{,}881$ ve $H(S_w) = 0{,}081$ yazar. Python ile: $H(S_b) = 0{,}881$, $H(S_w) = 0{,}081$, $H = 0{,}241$ bit.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.